Practical Week-5

Task 1: Implement a producer-consumer communication system using anonymous pipes where the parent process generates data and the child process consumes it. Measure the communication efficiency.

```
kiran@k:~$ cd ~
kiran@k:~$ mkdir ProducerConsumer
kiran@k:~$ cd ProducerConsumer
kiran@k:~/ProducerConsumer$ nano producer_consumer.c
kiran@k:~/ProducerConsumer$ gcc producer_consumer.c -o producer_consumer
kiran@k:~/ProducerConsumer$ ls
producer_consumer  producer_consumer.c
kiran@k:~/ProducerConsumer$ ./producer_consumer
Consumer: Received 400000 bytes

Producer: Sent 400000 bytes
Communication time: 0.001599 seconds
Data transferred: 0.38 MB
Communication throughput: 238.63 MB/s
kiran@k:~/ProducerConsumer$
```

kiran@k: ~/ProducerConsum

GNU nano 8.7.1 producer_consumer.c

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define DATA_SIZE 100000

int main()
{
    int pipefd[2];
    pid_t pid;

    int data[DATA_SIZE];

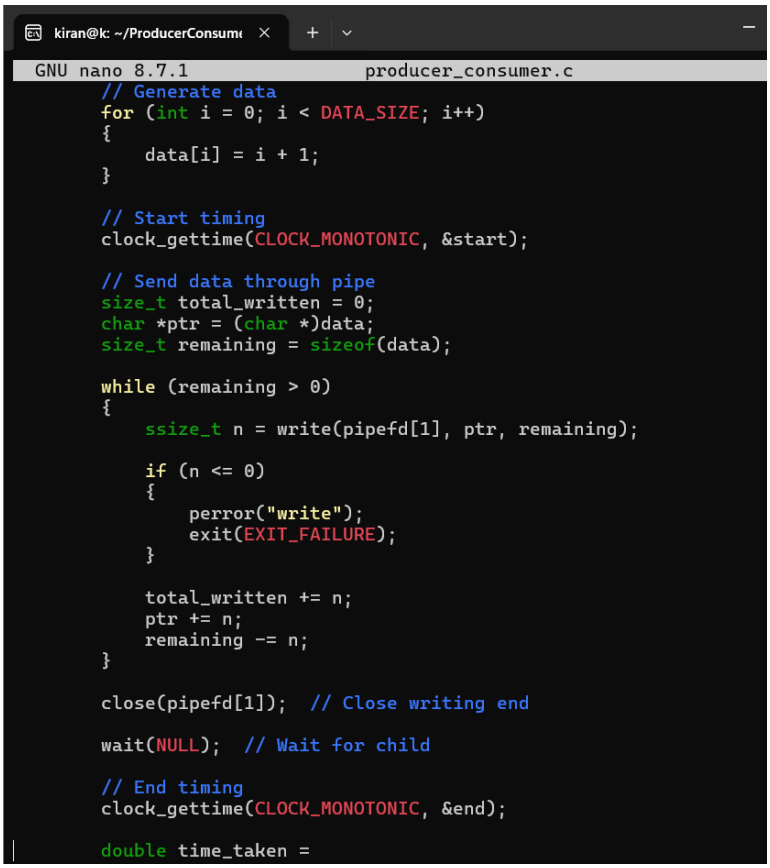
    struct timespec start, end;

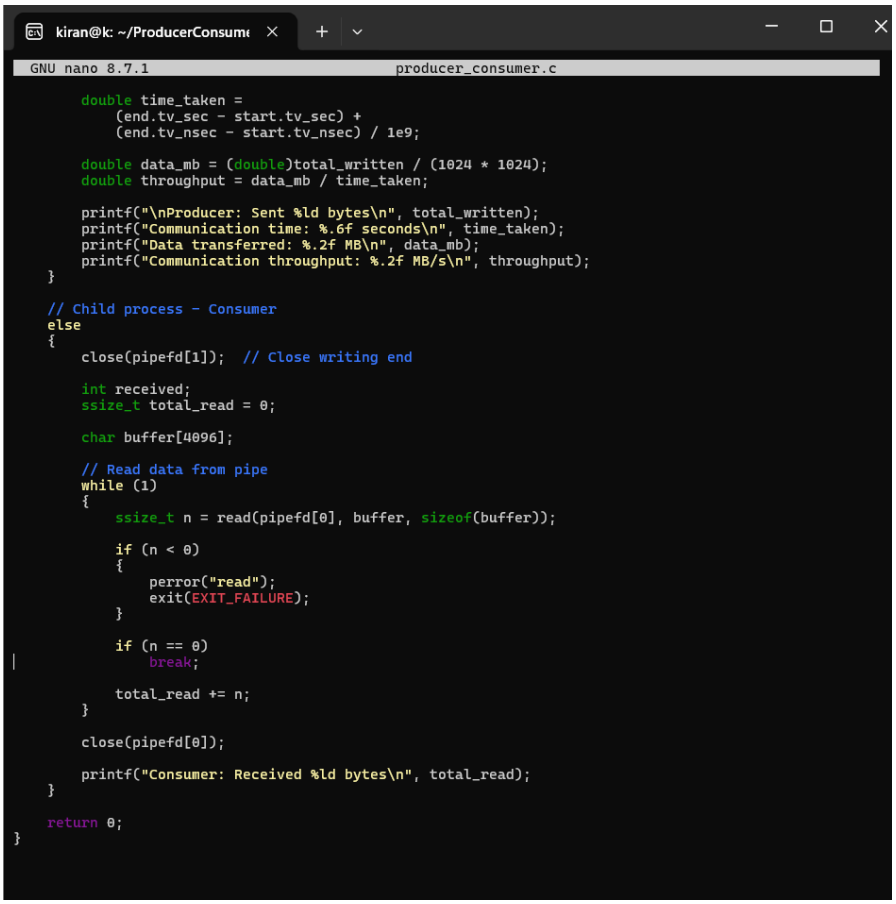
    // Create anonymous pipe
    if (pipe(pipefd) == -1)
    {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    // Create child process
    pid = fork();

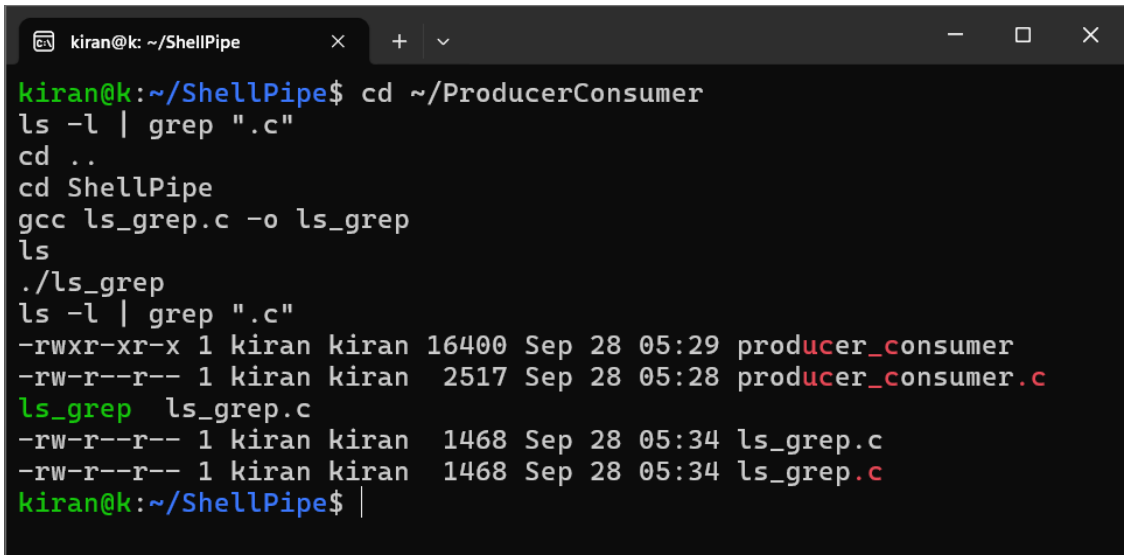
    if (pid < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

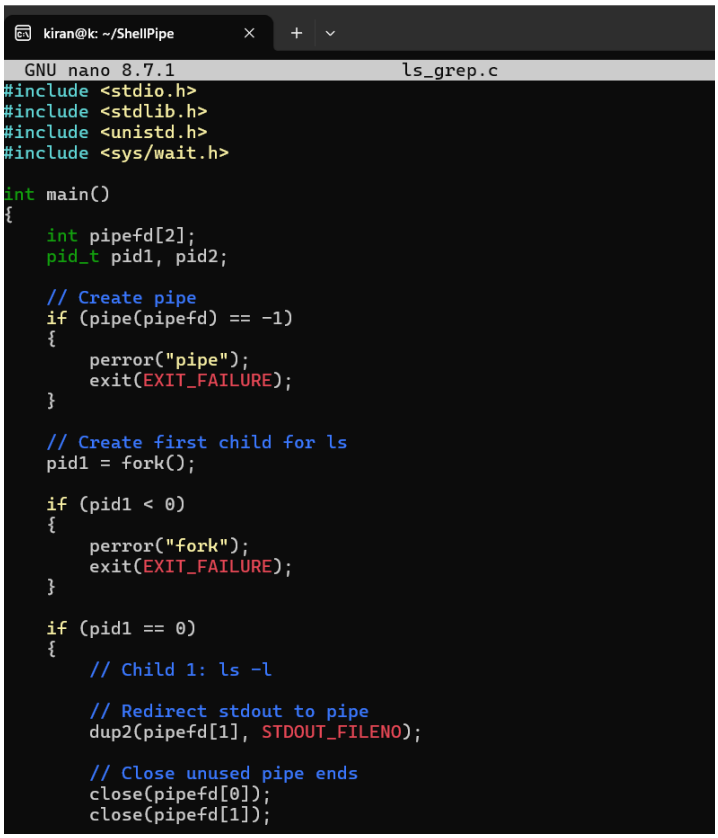
    // Parent process - Producer
    if (pid > 0)
    {
        close(pipefd[0]);  // Close reading end
```


kiran@lc: ~/ProducerConsume
GNU nano 8.7.1 producer_consumer.c
// Generate data
for (int i = 0; i < DATA_SIZE; i++)
{
data[i] = i + 1;
}
// Start timing
clock_gettime(CLOCK_MONOTONIC, &start);
// Send data through pipe
size_t total_written = 0;
char *ptr = (char *)data;
size_t remaining = sizeof(data);
while (remaining > 0)
{
ssize_t n = write(pipefd[1], ptr, remaining);
if (n <= 0)
{
perror("write");
exit(EXIT_FAILURE);
}
total_written += n;
ptr += n;
remaining -= n;
}
close(pipefd[1]); // Close writing end
wait(NULL); // Wait for child
// End timing
clock_gettime(CLOCK_MONOTONIC, &end);
double time_taken =



Task2: Develop a program that executes the equivalent of the shell command: ls -l | grep ".c" using fork(), pipe(), dup2(), and exec() system calls.


kiran@k: ~/ShellPipe
kiran@k:~/ShellPipe$ cd ~/ProducerConsumer
ls -l | grep ".c"
cd ..
cd ShellPipe
gcc ls_grep.c -o ls_grep
ls
./ls_grep
ls -l | grep ".c"
-rwxr-xr-x 1 kiran kiran 16400 Sep 28 05:29 producer_consumer
-rw-r--r-- 1 kiran kiran 2517 Sep 28 05:28 producer_consumer.c
ls_grep ls_grep.c
-rw-r--r-- 1 kiran kiran 1468 Sep 28 05:34 ls_grep.c
-rw-r--r-- 1 kiran kiran 1468 Sep 28 05:34 ls_grep.c
kiran@k:~/ShellPipe$


kiran@k: ~/ShellPipe
GNU nano 8.7.1 ls_grep.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main()
{
int pipefd[2];
pid_t pid1, pid2;
// Create pipe
if (pipe(pipefd) == -1)
{
perror("pipe");
exit(EXIT_FAILURE);
}
// Create first child for ls
pid1 = fork();
if (pid1 < 0)
{
perror("fork");
exit(EXIT_FAILURE);
}
if (pid1 == 0)
{
// Child 1: ls -l
// Redirect stdout to pipe
dup2(pipefd[1], STDOUT_FILENO);
// Close unused pipe ends
close(pipefd[0]);
close(pipefd[1]);

kiran@k: ~/ShellPipe

GNU nano 8.7.1 ls_grep.c

```c
        // Execute ls -l
        execlp("ls", "ls", "-l", (char *)NULL);

        // If execlp fails
        perror("execlp ls");
        exit(EXIT_FAILURE);
    }

    // Create second child for grep
    pid2 = fork();

    if (pid2 < 0)
    {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid2 == 0)
    {
        // Child 2: grep ".c"

        // Redirect stdin from pipe
        dup2(pipefd[0], STDIN_FILENO);

        // Close unused pipe ends
        close(pipefd[1]);
        close(pipefd[0]);

        // Execute grep ".c"
        execlp("grep", "grep", ".c", (char *)NULL);

        // If execlp fails
        perror("execlp grep");
        exit(EXIT_FAILURE);
    }

    // Parent process
```

